

Vers une formalisation adaptée à la détection d'anomalie multi-domaines

Contents

3.1	Objectifs et cadre de détection	2
3.2	État de l'art de la détection d'anomalies	3
3.2.1	Approche historique dans les réseaux traditionnels	3
3.2.2	Apprentissage profond appliqué aux réseaux 5G	4
3.2.3	Comparaison aux techniques des systèmes distribués	7
3.2.4	Synthèse	8
3.3	Méthodologie de formalisation	9
3.3.1	Extraction des features depuis la payload applicative	9
3.3.2	Transformation en graph	10
3.3.3	Encodage des arcs et nœuds	12
3.4	Détection d'anomalies dans des graphs	13
3.4.1	Introduction aux GNN	13
3.4.2	Etat de l'art des GNN	15
3.4.3	Formalisation adapté à l'utilisation de GNN dynamiques	19
3.4.4	Fonctionnement détaillé des GNN dynamique	20
3.4.5	Adaptation à notre cas d'utilisation	22
3.5	Encodage des paramètres de dimension non-finie via NLP	25
3.5.1	Tour d'horizon des techniques de NLP	26
3.5.2	Mise en place de l'encodage sémantique	29
3.5.3	Avantages et limites	30
3.6	Conclusion	30

Le passage aux réseaux 5G s'accompagne d'un changement de paradigme majeur, caractérisé par une architecture distribuée et une complexification des interactions entre entités. Par ailleurs, l'émergence d'anomalies sémantique, séquentielle et topologique impose de nouvelles exigences en matière d'analyse. Dans ce contexte, il devient nécessaire non seulement d'adopter une analyse fine au niveau des paquets, mais également de prendre en compte le contexte dans

lequel s'inscrivent les échanges. Or, cette problématique reste relativement récente dans le domaine des télécommunications, et les méthodes actuelles de détection d'anomalies apparaissent inadaptées à ce nouveau paradigme.

Dans ce contexte, nous proposons de représenter les échanges au sein des réseaux 5G sous la forme d'un graphe dynamique, permettant de capturer les interactions entre les différentes entités du réseau ainsi que l'évolution de ces interactions dans le temps. Cette formalisation nous permet d'exploiter des techniques issues du domaine des GNN, intrinsèquement adaptées à l'analyse des interactions complexes. Nous nous intéressons plus particulièrement aux GNN dynamiques, qui intègrent nativement la dimension temporelle.

3.1 Objectifs et cadre de détection

Dans nos travaux, nous nous plaçons du point de vue de l'opérateur d'un réseau 5G et nous nous intéressons à la détection d'anomalies en nous concentrant exclusivement sur le trafic de contrôle échangé entre les entités du réseau, qu'il s'agisse du cœur de réseau ou du RAN. Le trafic utilisateur n'est pas pris en compte, car il ne présente pas de spécificités propres à l'architecture 5G et ne relève pas de la responsabilité directe du fournisseur d'accès réseau.

En se plaçant du point de vue de l'opérateur, nous supposons que celui-ci possède l'ensemble des NF et dispose d'un accès complet au trafic de contrôle. Dans l'architecture 5G, la fonction Service Communication Proxy (SCP) agit comme un intermédiaire dans les échanges entre les différentes NF, notamment pour le routage des requêtes et la vérification des autorisations d'accès. Dans ce contexte, il est raisonnable de considérer que l'opérateur peut observer l'intégralité du trafic de contrôle en déployant des sondes au niveau des SCP, qui constituent des points d'observation privilégiés des interactions entre NF.

Ce trafic est généralement chiffré, par exemple via TLS sur le SBI ou via IPsec pour certains canaux, tels que N3 ou N4, reposant sur des protocoles spécifiques à la 5G. Cependant, en adoptant le point de vue de l'opérateur, nous supposons disposer des clés de chiffrement nécessaires pour accéder au contenu de ces échanges en clair.

L'objectif serait de pouvoir exécuter nos modèles de détection en temps réel. Malgré les débits élevés offerts par la 5G, le volume de données est majoritairement porté par le trafic utilisateur, qui n'entre pas dans le périmètre de cette étude. Le trafic de contrôle, lui, est principalement composé de messages liés aux changements d'état des entités du réseau, qu'il s'agisse d'UE ou de NF, et ont une fréquence d'occurrence plus modérée. Par conséquent, bien que le surcoût induit par nos modèles puisse avoir un impact sur les performances, il est raisonnable de supposer que les ressources disponibles sont suffisantes pour permettre un traitement en temps réel.

La remédiation des anomalies détectées ne relève pas du périmètre de ce travail et est considérée comme une brique fonctionnelle distincte. Nous nous concentrons

exclusivement sur la détection et les alertes générées par notre système ont vocation à être exploitées de manière similaire à celles produites par un IDS classique. Elles peuvent ainsi conduire, par exemple, à des ajustements des règles de firewall ou à des modifications de l'architecture réseau.

L'objectif de ces travaux n'est pas de concevoir un système de détection absolu et entièrement autonome. En effet, une anomalie ne constitue pas nécessairement le signe d'une activité malveillante et peut résulter d'une mise à jour, d'une erreur de configuration ou encore d'une interconnexion avec un système tiers, comme dans le cas du roaming. Dans ce contexte, la distinction entre une anomalie bénigne et une attaque de type zero-day requiert in fine une expertise humaine. Par conséquent, l'un des aspects essentiels de nos travaux est de proposer un système interprétable, venant assister une expertise humaine plutôt que de la remplacer.

3.2 État de l'art de la détection d'anomalies

Cette section propose un panorama des travaux de recherche consacrés à la détection d'anomalies. Dans un premier temps, nous présenterons les limites des approches traditionnelles appliquées à notre problématique, en montrant que les objectifs visés, les contraintes opérationnelles ainsi que les zones d'observation diffèrent sensiblement de notre contexte d'étude. Nous nous intéresserons ensuite aux méthodes développées pour la détection d'anomalies dans les réseaux 5G, afin d'évaluer l'état actuel de la recherche sur notre problématique spécifique. Nous pourrions ainsi identifier les limites ainsi que les perspectives encore peu explorées. Enfin, nous adopterons une perspective plus large en examinant les travaux issus du domaine des systèmes distribués qui, comme nous l'avons montré précédemment, présentent de nombreuses similarités structurelles et fonctionnelles avec les architectures 5G.

3.2.1 Approche historique dans les réseaux traditionnels

La détection d'attaques au sein des réseaux est historiquement assurée par des IDS. Ces systèmes reposent généralement sur une analyse du trafic agrégé sous forme de flux, plutôt qu'au niveau des paquets individuels. Cette approche permet d'extraire des métriques globales et d'effectuer des analyses de volumétrie, par exemple afin de détecter des variations significatives du volume de trafic ou repérer des signatures d'attaques connues.

Parmi les IDS les plus connus on retrouve Snort [?], Zeek [?] et Suricata [?], qui reposent sur des règles prédéfinies pour identifier des motifs (aussi appelés signatures) d'attaques spécifiques dans le flux réseau. Ce sont la plupart du temps des outils open source qui bénéficient d'une large communauté qui contribue à l'enrichissement de leur base de signatures.

Des approches fondées sur l'apprentissage automatique supervisé ont également été proposées pour la détection d'intrusion, notamment à l'aide de réseaux bayésiens [?], de machines à vecteurs de support (SVM) [?] ou encore d'arbres de

décision [?]. Ces méthodes présentent l'avantage d'être relativement transparentes, interprétables et efficaces pour détecter des attaques déjà connues.

Parmi les approches non-supervisées, Münz et al. [?] proposent, par exemple, de découvrir des motifs anormaux en adaptant l'algorithme K-Means au trafic réseau, permettant ainsi de regrouper les données en fonction de leur similarité sans nécessiter d'étiquettes. Néanmoins, les caractéristiques utilisées en entrée restent relativement limitées et se restreignent notamment au nombre de paquets, au volume d'octets ainsi qu'aux adresses source et destination. Une telle approche permet d'identifier des motifs anormaux liés à la volumétrie. Toutefois, elle ne prend en compte ni le contenu sémantique des paquets ni le contexte des échanges.

Ariu et al. [?] proposent quant à eux d'utiliser des modèles de Markov cachés afin de modéliser le trafic réseau sous forme de séquences d'octets. Ces modèles apprennent des probabilités de transition à partir de trafic bénin, puis évaluent dans quelle mesure un trafic observé s'en écarte. Cependant, les séquences considérées sont limitées à des fenêtres de taille relativement réduite, ce qui empêche la capture de dépendances temporelles à long terme. De plus, cette approche ne permet pas de modéliser explicitement les interactions entre paquets.

Cette étude met en évidence le décalage entre les approches historiques de détection d'intrusion et les besoins spécifiques liés aux réseaux 5G. Les méthodes traditionnelles, principalement conçues pour l'analyse agrégée de flux, se révèlent peu adaptées à un environnement distribué et softwarisé. Ainsi, les IDS traditionnels restent généralement limités, soit par le manque d'expressivité des caractéristiques considérées, soit par une modélisation insuffisante des dépendances temporelles et des interactions complexes entre paquets. Ces limites justifient ainsi la nécessité de développer des méthodes plus adaptées aux spécificités des architectures 5G.

3.2.2 Apprentissage profond appliqué aux réseaux 5G

Après avoir présenté les limites des approches traditionnelles de détection d'intrusion, nous nous intéressons désormais aux méthodes spécifiquement conçues pour les réseaux 5G. L'objectif est d'analyser dans quelle mesure ces travaux prennent en compte les dimensions sémantique, séquentielle et topologique. Cette analyse permettra d'identifier les avancées récentes du domaine, mais également les limitations persistantes des approches actuelles.

Parmi les travaux de détection spécifiques à la 5G, Kale et al. [?] proposent d'utiliser des réseaux de neurones convolutionnels (CNN) en considérant les octets d'un paquet comme une image sur laquelle sont ensuite étudiées des variations structurelles. Les modèles CNN obtiennent de très bonnes performances dans de nombreux domaines, y compris dans des tâches qui ne relèvent pas intrinsèquement de la vision par ordinateur. Toutefois, comme ils n'interprètent pas le contenu des données, leur capacité de détection se limite à un sous-ensemble d'attaques sémantiques dans lesquelles les valeurs s'écartent fortement du trafic normal. Ils ne permettent donc pas de détecter des variations légères de valeurs d'identifiants

comme les adresses IP ou les identifiants utilisateurs (IMSI). De plus, le travail présenté par Kale et al. se concentre uniquement sur la détection sémantique et ne prend pas en compte le contexte des échanges. Lam et al. [?] ajoutent un contexte temporel à leur CNN en alimentant ses sorties dans un réseau de neurones récurrent (RNN). Cependant, en plus de ne pas être satisfaisant sur le domaine sémantique, leur modèle ne prend pas en compte l'identité des acteurs impliqués dans les échanges et n'est donc pas capable de détecter des attaques relationnelles.

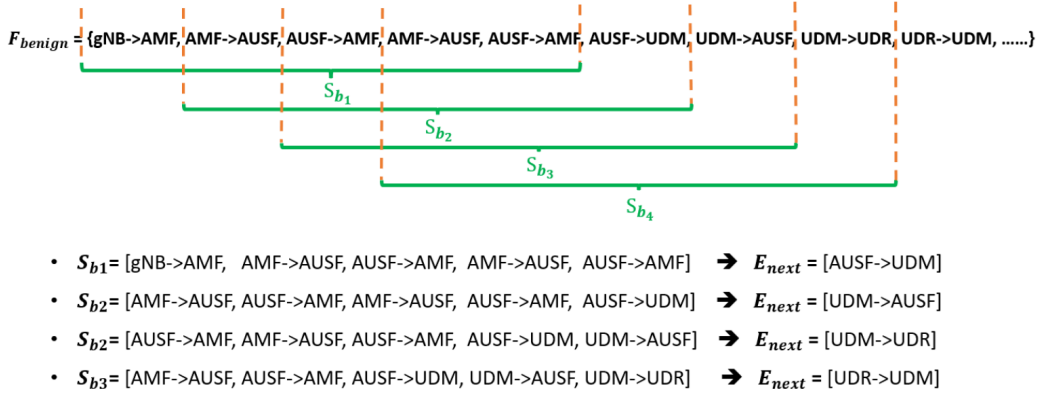


Figure 3.1: Représentation d'une séquence d'événement bénins entre différentes entités tel que présenté par ADSeq-5GCN [?].

AutoGuard [?] et ADSeq-5GCN [?] modélisent les échanges réseau sous forme de séquences d'interactions entre entités du réseau tel que représenté sur figure ???. Ces modèles utilisent ensuite des RNN afin de capturer les dépendances temporelles et prédire les interactions futures. Leurs approches intègrent l'identité des entités communicantes, ce qui leur permet de prendre en compte, dans une certaine mesure, le contexte topologique des interactions. Cependant, elles rencontrent des difficultés à modéliser efficacement les dépendances relationnelles lorsque les interactions sont éloignées dans le temps. De plus, le principal problème est que leur formalisme n'intègre pas le contenu des paquets et ne considère donc aucunement la dimension sémantique. 5GCCGuard [?] tente de pallier cette limitation en enrichissant leurs séquences par l'intégration de l'URL associée à chaque message. Bien qu'elle apporte une information sémantique supplémentaire, cette approche demeure très limitée car l'information propre à l'URL ne représente qu'une fraction du contenu réel des paquets et n'est applicable qu'au trafic HTTP.

Prov5GCC [?] propose de son côté de représenter le trafic sous forme de graphe de provenance. Dans cette représentation, illustrée sur la figure ??, les nœuds correspondent aux entités du réseau et chaque message échangé entre ces entités est modélisé par une arête, à laquelle est associé le contenu du paquet. La formalisation en graphe est particulièrement adaptée à l'analyse topologique des interactions du réseau. Cependant, elle ne modélise pas explicitement la dimension temporelle des échanges, ce qui restreint sa capacité à capturer l'évolution dynamique des

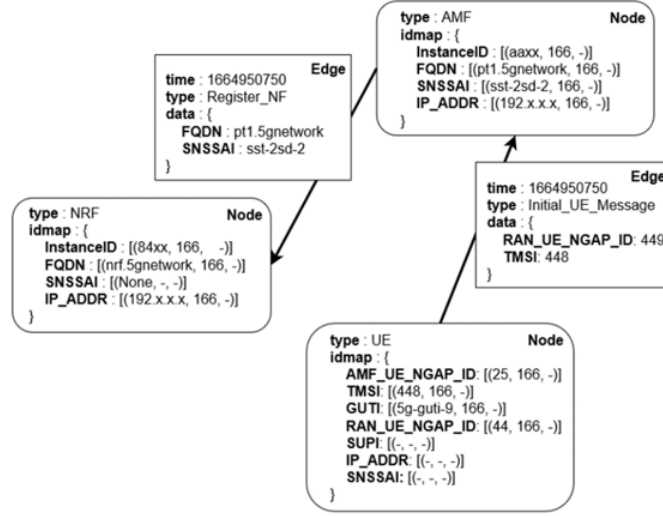


Figure 3.2: Trafic de contrôle 5G formalisé par Prov5GC [?] en tant que graphe de provenance

interactions. De plus, le fait d'agréger le payload au niveau des arêtes limite fortement l'exploitation de l'information contenue dans les paquets. Dans leur cas la payload est utilisée par un algorithme de détection heuristique spécifique à chaque attaque, ce qui rend l'approche relativement lourde et peu adaptée à des environnements complexes.

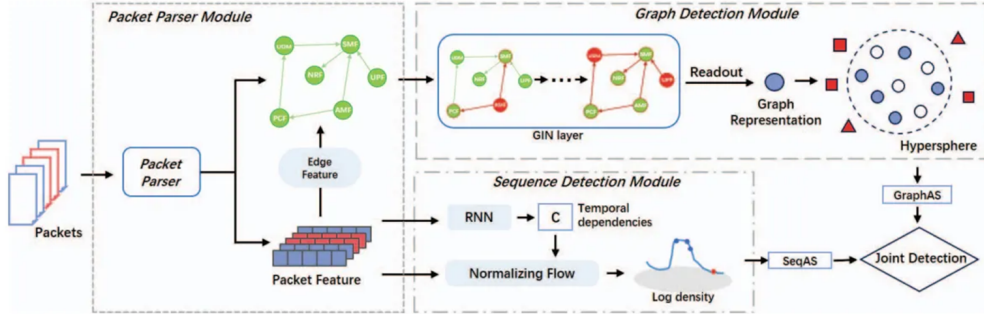


Figure 3.3: Pipeline de GSAD [?] représentant leur formalisation ainsi que les trois modules de détections traitant respectivement les dimensions sémantique, séquentielle et topologique

GSAD [?] propose également de modéliser les données sous forme de graphe de manière similaire à Prov5GC [?]. La principale différence réside dans la représentation du contenu des arêtes. Au lieu de l'encoder sous forme de dictionnaire monolithique, les auteurs le transforment en flux d'octets structuré comme une image. Cette représentation leur permet d'utiliser un CNN pour encoder chaque paquet et analyser sa dimension sémantique. Les représentations sémantiques

sont ensuite injectées dans un GNN afin de modéliser les interactions entre les paquets, puis dans un RNN pour capturer les dépendances temporelles. À notre connaissance, GSAD [?] est la seule approche de détection d’anomalies 5G qui considère simultanément les trois dimensions de détection. La principale limite de leur approche réside dans le fait que, comme évoqué précédemment, les CNN ne permettent qu’une analyse relativement superficielle de la dimension sémantique des paquets. La Figure ?? illustre l’architecture générale du pipeline proposé par GSAD et met en évidence les trois composantes du modèle correspondant respectivement aux trois dimensions.

Référence	Sémantique	Séquentiel	Topologique
Kale et al. [?]	Oui	Non	Non
Lam et al. [?]	Oui	Oui	Non
AutoGuard [?]	Non	Oui	Partiel
ADSeq-5GCN [?]	Non	Oui	Partiel
5GCGuard [?]	Non	Oui	Partiel
Prov5GC [?]	Partiel	Non	Partiel
GSAD [?]	Partiel	Partiel	Oui

Table 3.1: Comparaison des approches de l’état de l’art de détection d’anomalies 5G selon les dimensions considérées

Le tableau ?? résume les différentes approches de détection d’anomalies dans les réseaux 5G, en mettant en évidence les dimensions sémantique, temporelle et topologique prises en compte par chacune d’elles. L’état de l’art montre que peu d’approches considèrent simultanément les trois dimensions. À travers les travaux tels que GSAD[?] et Prov5GC[?], les GNN semblent particulièrement adaptés pour modéliser les interactions entre entités. À l’inverse, aucune des modélisations proposées ne semble réellement adaptée au traitement du contenu sémantique des paquets, celui-ci étant le plus souvent considéré comme un bloc monolithique.

3.2.3 Comparaison aux techniques des systèmes distribués

Enfin, nous prenons un peu de recul en nous intéressant aux travaux issus du domaine des systèmes distribués. Comme évoqué précédemment, les architectures 5G changent de paradigme et présentent maintenant de nombreuses similarités avec ces systèmes fortement distribués et dynamiques. Nous examinons donc dans cette section les principales approches de détection d’anomalies proposées dans ce domaine, afin d’identifier dans quelle mesure elles exploitent les spécificités de ces architectures et les enseignements qu’il est possible d’en tirer pour notre problématique.

Parmi les approches exploitant la nature distribuée des infrastructures, on retrouve notamment des travaux basés sur le Federated Learning (FL). DioT[?] et VFL[?], par exemple, promeuvent les avantages de la fédération dans les systèmes distribués, notamment en termes de passage à l’échelle, de confidentialité et de

décentralisation de l'apprentissage. Cependant, leur contribution se concentre principalement sur le mécanisme de fédération lui-même. Les modèles utilisés restent relativement classiques et ne couvrent généralement qu'une partie des dimensions nécessaires à une détection complète des anomalies. DioT[?] s'appuie ainsi sur des Recurrent Neural Network (RNN) de type Gated Recurrent Unit (GRU) pour modéliser les dépendances temporelles, tandis que VFL[?] exploite GraphSAGE[?] afin de capturer les relations structurelles entre entités. Dans ce contexte, le FL apparaît davantage comme une extension permettant de rendre la détection scalable et réaliste d'un point de vue opérationnel que d'une méthode de détection à part entière.

Dans la littérature des systèmes distribués, on retrouve également de nombreuses approches fondées sur l'analyse de logs systèmes, telles que [?, ?]. Toutefois, ces méthodes présentent plusieurs limitations. Bien qu'elles permettent d'observer l'activité du système à un niveau relativement élevé, les logs constituent une représentation indirecte et partielle des événements réseau. Ils dépendent fortement des implémentations logicielles et peuvent contenir du bruit, des informations manquantes ou des incohérences. Ainsi, certains comportements anormaux peuvent ne jamais être journalisés, ce qui limite intrinsèquement la capacité de détection. Une piste complémentaire consiste à combiner différentes sources d'observation, comme proposé par Bogatinovski et al.[?], en fusionnant logs et traces afin d'enrichir la représentation des événements. Cependant, cette approche introduit également une complexité supplémentaire dans la gestion et la corrélation des données, et peut conduire à une surcharge informationnelle. Dans notre cas, les paquets réseau fournissent déjà une observation suffisamment riche et fine du système. Ainsi, l'ajout de sources supplémentaires risquerait alors davantage de brouiller l'information pertinente que de réellement améliorer la détection.

Enfin, une part importante des travaux de la littérature s'intéresse aussi aux GNN [?, ?] et plus spécifiquement sur des approches dynamiques[?]. Cette convergence entre les domaines de la 5G et des systèmes distribués confirme que la modélisation par graphes constitue un cadre particulièrement pertinent pour capturer les interactions complexes en intégrant à la fois les dimensions structurelles et temporelles.

3.2.4 Synthèse

L'analyse de la littérature sur la détection d'anomalies réseau met en évidence que les approches traditionnelles ne sont pas adaptées à notre problématique, principalement en raison d'un niveau de granularité trop élevé qui empêche la capture de comportements subtils. En se focalisant sur des travaux spécifiquement dédiés à la 5G, on constate par ailleurs qu'aucune approche ne propose à ce jour un formalisme unifié permettant de prendre en compte simultanément les trois dimensions de détection définies dans ce travail. Néanmoins, plusieurs éléments issus de l'état de l'art convergent vers des directions prometteuses. En particulier, des travaux tels que GSAD, ainsi que les approches issues des systèmes distribués,

montrent que les représentations par graphes et les GNN font désormais l'objet d'un consensus croissant. Ces modèles apparaissent comme des outils particulièrement adaptés pour représenter la structure complexe des interactions et pourraient constituer une base solide pour répondre à notre problématique.

3.3 Méthodologie de formalisation

Nous avons vu que le formalisme en graphe proposé par GSAD consiste à regrouper l'ensemble du contenu des paquets dans un bloc monolithique associé à une arête, ce qui limite fortement la capacité à exploiter correctement la dimension sémantique. Dans ce contexte, nous proposons une représentation plus fine du trafic, en décomposant le contenu des paquets en nœuds indépendants correspondant chacun à un paramètre différent. Nous détaillons à présent notre méthodologie de formalisation. Nous commencerons par l'extraction des caractéristiques à partir du trafic de contrôle, puis nous présenterons leur transformation en graphes.

3.3.1 Extraction des features depuis la payload applicative

Comme notre objectif est principalement d'analyser le contenu applicatif des paquets, nous extrayons la payload applicative pour chaque paquet. Les seules informations non applicatives conservées concernent la couche IP, utilisées comme identifiant afin de discriminer les différentes entités du réseau. Cette approche n'est pas idéale, car elle est sensible au *spoofing*. Toutefois, dans un cadre opérationnel, ces adresses pourraient être remplacées par des identifiants plus robustes, tels que des empreintes ou des hash, qui pourraient représenter les entités de manière unique.

Pour récupérer le contenu des payloads, nous utilisons *Scapy*, qui permet de parser les paquets réseau et de générer, pour chacun d'eux, une représentation sous forme de dictionnaire associant les noms des paramètres à leurs valeurs respectives.

Une des problématiques du trafic applicatif est la présence d'imbrication dans le contenu des données de la payload. En effet, les paquets HTTP/2 contiennent couramment des structures JSON avec des données hiérarchisées. Conserver explicitement le chemin d'imbrication dans une modélisation en graphe nécessiterait d'introduire de nouveaux types de nœuds et donc de diluer l'information produite par le contenu lui-même. Or, même si l'imbrication est utile pour différencier des paramètres, elle reste secondaire par rapport aux valeurs elles-mêmes.

Pour gérer ce problème, nous avons donc choisi d'aplatir les dictionnaires extraits des payloads en construisant le nom de chaque paramètre de feuille comme la concaténation de ses clés précédentes, séparées par des délimiteurs. On obtient ainsi des identifiants de paramètres composés, par exemple `http2.headers.content-type`, permettant de désigner le champ `content-type` d'un paquet HTTP/2. Cette approche permet de conserver l'information d'imbrication en tant que nom de paramètre tout en maintenant une structure suffisamment simple pour notre formalisation en graphe.

3.3.2 Transformation en graph

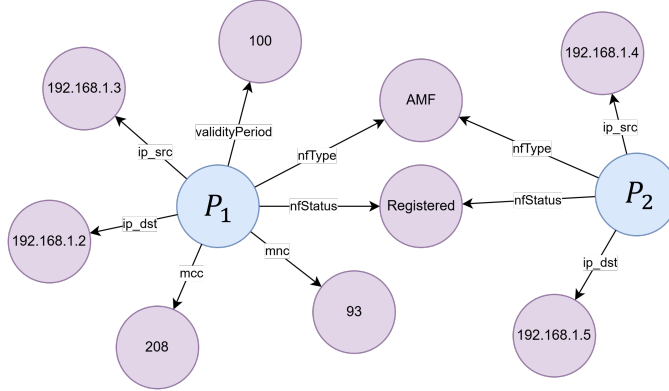


Figure 3.4: Exemple de transformation de paquets en sous-graphes. Les nœuds mauves représentent les valeurs des paramètres, tandis que les nœuds bleus représentent les paquets eux-mêmes.

Au lieu de représenter un paquet comme une arête entre deux entités, nous le modélisons comme un sous-graphe, où chaque paramètre est un nœud. Chacun de ces nœuds paramètre est relié à un nœud central qui représente le paquet. Chaque nœud paramètre contient la valeur du paramètre, tandis que l'arête reliant ce nœud au nœud central contient l'information du nom du paramètre correspondant. Lors de la création d'un sous-graphe représentant un paquet, de nouveaux nœud paramètres sont créés seulement s'il n'existaient pas encore dans le graphe global. Dans le cas contraire, le nœud existant est réutilisé. Ce mécanisme permet de relier progressivement les différents sous-graphes entre eux à travers leurs paramètres partagés. La figure ?? illustre le processus de transformation de deux paquets en deux sous-graphes. Ces deux paquets partagent certains paramètres et leurs sous-graphes respectifs se retrouvent donc connectés.

Dans notre formalisme nous avons donc deux types de nœuds, paramètres et centraux qui ne peuvent jamais être reliés directement à d'autres nœuds du même type. L'ajout de ces nœuds correspond à la réception de nouveaux paquets. On a donc un graphe biparti purement additif. La réception d'un paquet ne peut entraîner que l'ajout de nouveaux nœuds et d'arêtes, sans jamais modifier ni supprimer ceux déjà présents.

Cette décomposition permet de connecter des attributs issus de paquets différents en s'appuyant sur des caractéristiques communes, plutôt que seulement sur l'identité des participants. Une telle représentation offre un niveau de granularité très fin et permet de capturer des interactions plus riches entre les caractéristiques des paquets. Enfin, cette approche permet également d'identifier précisément les paramètres à l'origine d'un comportement anormal et se révèle donc, par construction, hautement interprétable.

Nous allons maintenant analyser la représentation des anomalies de chaque

dimension une fois adaptés à notre formalisme.

3.3.2.1 Anomalies sémantiques

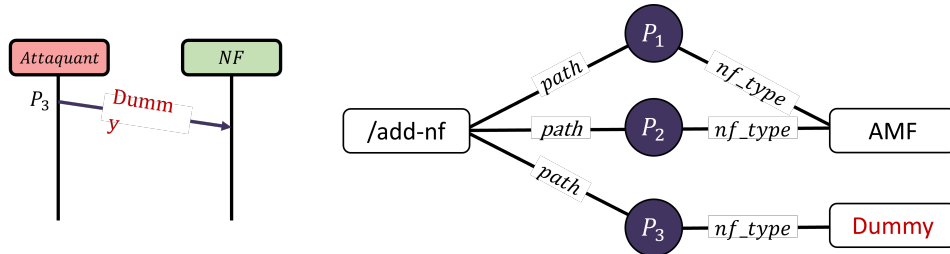


Figure 3.5: Anomalie sémantique et sa représentation sous forme de graphe

La figure ?? illustre une anomalie sémantique dans laquelle un champ inhabituel apparaît. Dans le cas présent, il s’agit du paquet P3 dont la valeur du champs *nf-type* n’existe pas en 5G. Cette anomalie peut être détectée en le comparant aux autres paquets de même nature. On observe par exemple que les paquets P1 et P2, qui sont, comme P3, des requêtes vers l’endpoint *add-nf*, possèdent des valeurs identiques pour le champ *nf-type*, alors que P3 s’en écarte. Un modèle de détection ayant appris la structure usuelle de ce type de paquet pourrait ainsi identifier l’anomalie en observant uniquement le sous-graphe associé à P3.

3.3.2.2 Anomalies topologiques

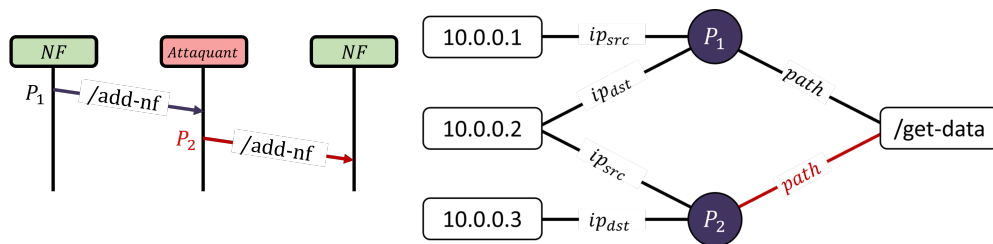


Figure 3.6: Anomalie topologique et sa représentation sous forme de graphe

Sur la figure ??, on observe un exemple d'attaque de type man-in-the-middle, où un paquet P1, initialement envoyé à une première adresse, génère l'envoi d'un nouveau paquet P2 identique (ici simplifié par le fait qu'il s'agisse du même endpoint). Ce nouveau paquet a comme source la destination du paquet précédent et comme destination une nouvelle adresse. Ce cas ne peut normalement pas survenir en 5G, car il n'existe aucun scénario légitime de redirection de trafic de ce type. Le paquet P2 est une copie de P1 et paraît donc légitime pris individuellement. Il est ici nécessaire de prendre en compte le contexte de redirection afin de détecter l'anomalie.

3.3.2.3 Anomalies séquentielles

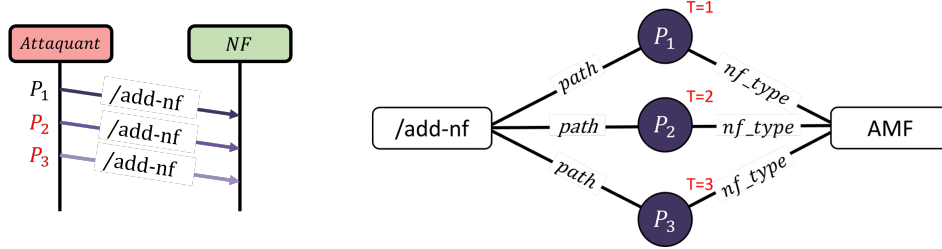


Figure 3.7: Anomalie séquentielle et sa représentation sous forme de graphe

Enfin, sur la figure ??, on observe un exemple d'anomalie séquentielle. Dans ce cas, le contenu et les interactions des paquets sont normaux et la caractéristique anormale concerne uniquement leur ordre d'arrivée. Le problème est que le formalisme actuel ne modélise pas explicitement la dimension temporelle. Il serait possible de l'intégrer en ajoutant à chaque sous-graphe un nœud représentant le temps d'observation du paquet, mais avec cette approche la dimension temporelle serait confondue avec les autres paramètres alors qu'elle est particulièrement importante. Il est donc nécessaire de proposer une modélisation permettant d'intégrer le temps de manière plus explicite.

3.3.3 Encodage des arcs et nœuds

Pour être exploitables par des modèles de détection, le contenu des nœuds et des arêtes doivent être encodés sous forme de vecteurs numériques. Pour les arrêtes, le vecteur est construit à partir du nom du paramètre porté par l'événement, lequel est toujours de type textuel. L'espace des noms de paramètres possibles, le vocabulaire, est considéré comme statique car il dépend de l'implémentation de l'infrastructure 5G au moment de l'entraînement du modèle. Ainsi, si le jeu de données d'entraînement est suffisamment exhaustif, il est possible de couvrir la majorité des noms de paramètres susceptibles d'apparaître. Cependant, lorsqu'un paramètre inconnu est rencontré, deux cas peuvent être envisagés. Il peut s'agir d'une évolution de l'implémentation, auquel cas des anomalies peuvent être observées de manière légitime tant que le modèle n'a pas été ré-entraîné. Il peut également s'agir d'une tentative d'attaque, par exemple de fuzzing, qui exploite des appels API malformés et introduisant des noms de paramètres inexistantes. Dans ce cas nous avons une anomalie sémantique sur le nom des paramètres. Comme notre vocabulaire est ici fixe, l'ensemble des valeurs d'arêtes peut être représenté par des vecteurs uniques auquel on ajoute un vecteur dédiés aux valeurs inconnues. Les méthodes d'encodage textuel utilisables pour cela sont détaillées dans la section suivante.

Pour les nœuds, le vecteur peut être construit à partir de la valeur du paramètre associé. Ces valeurs peuvent correspondre aussi bien à des variables catégorielles, comme des méthodes HTTP, qu'à des identifiants uniques, comme des adresses

IP. L'analyse de l'échantillon collecté dans notre jeu de données montre qu'environ 90% des valeurs de paramètres correspondent à des identifiants uniques. Si les données ne contenaient que des variables catégorielles, un encodage classique de type one-hot aurait pu être utilisé. Toutefois, les identifiants sont générés au cours de l'exécution du système et ne peuvent donc pas être exhaustivement connus lors de l'entraînement. Le vocabulaire est donc dynamique, ce qui rend l'approche one-hot inadaptée ici. Une solution pour gérer ce vocabulaire dynamique serait d'utiliser des méthodes issues du traitement automatique du langage naturel (NLP) afin de rapprocher sémantiquement les valeurs textuelles. Cependant, les identifiants, tels que les adresses IP, ne correspondent généralement pas à des mots porteurs de sens linguistique. Il faudrait alors définir des mécanismes d'encodage spécifiques pour chaque type d'identifiant, ce qui ne serait ni scalable ni cohérent avec la nature dynamique des infrastructures 5G. La solution la plus simple consiste donc à ne pas prendre en compte l'information portée par les valeurs des paramètres. Dans le cas des identifiants uniques, leur valeur intrinsèque présente de toute façon peu d'intérêt. De même, pour les variables catégorielles, la sémantique d'un nœud peut être inférée à travers l'analyse de la structure du graphe, comme illustré dans la figure ?? présentée précédemment.

Dans nos représentations, seule l'information portée par les arêtes sera donc conservée explicitement. L'information associée aux nœuds pourra quant à elle être reconstruite implicitement par le GNN à travers les convolutions successives et l'agrégation des informations de voisinage.

3.4 Détection d'anomalies dans des graphes

La détection d'anomalies à l'aide de GNN constitue un domaine de recherche en pleine expansion, connaissant un développement significatif ces quinze dernières années. Nous introduirons dans cette section les principes fondamentaux et les caractéristiques essentielles des GNN. Nous examinerons ensuite l'état de l'art des GNN puis. Enfin, nous verrons comment adapter notre formalisme aux graphes dynamiques.

3.4.1 Introduction aux GNN

Le fonctionnement des GNN est très proches de celui des CNN présentés précédemment. Dans les deux cas, le principe repose sur une agrégation locale de l'information, la principale différence étant le formalisme. Dans une image, cette agrégation s'effectue sur des voisinages de pixels, tandis que dans un graphe elle s'applique au voisinage des nœuds. Certains travaux proposent notamment de traiter une image comme un graphe, en considérant chaque pixel comme un nœud relié à ses voisins par un arc, et d'y appliquer des GNN. Néanmoins, cette approche reste relativement marginale [?].

L'idée fondamentale de ces deux types de modèles est que chaque itération du processus de convolution, appelé *Message Passing* pour les GNN, permet de diffuser



Figure 3.8: Illustration du processus de convolution. La première figure représente le graphe à son état initial. Dans la deuxième, une première convolution est appliquée. On observe que les informations des nœuds feuilles sont agrégées et propagées à leurs voisins directs. Dans la troisième, une deuxième convolution est réalisée, et les informations précédemment propagées sont à nouveau diffusées jusqu'au nœud central du graphe. Après ces deux convolutions, le nœud central contient une représentation de son voisinage complet à distance 2.

l'information aux voisins directs. En procédant de manière récursive, chaque couche de convolution propage cette information à une distance croissante, augmentant progressivement le champ de réception des représentations. La figure ?? illustre ce processus de diffusion de l'information réalisé par deux convolutions successives.

En apprentissage sur graphes, on distingue généralement les approches transductives et inductives selon les informations disponibles au moment de l'entraînement. En mode transductif, le modèle connaît déjà toutes les données du dataset, y compris les nœuds à prédire et leur voisinage, mais leurs labels restent inconnus. À l'inverse, l'apprentissage inductif suppose que les nœuds à traiter pendant le test ne sont pas visibles durant l'entraînement. Le modèle doit donc apprendre des règles générales capables de se transférer à de nouvelles structures. Le choix entre ces deux paradigmes dépend fortement de la tâche considérée. Mais, de par leur nature évolutive, l'apprentissage inductif est souvent privilégié dans les travaux sur les graphes dynamiques, car il permet de généraliser à des nœuds et interactions apparaissant au cours du temps.

Parmi les tâches courantes dans l'état de l'art des graphes, on retrouve principalement la classification, la régression et le clustering. Ces tâches peuvent être appliquées à différents niveaux de granularité, que ce soit sur des nœuds, des arêtes ou des graphes entiers. La classification consiste à attribuer un label discret à un élément du graphe, tandis que la régression vise à prédire une valeur continue associée à celui-ci. Le *clustering*, quant à lui, a pour objectif de regrouper les éléments du graphe selon leurs représentations apprises. On retrouve également la tâche de prédiction de lien, qui consiste à prédire l'apparition future d'arêtes entre des nœuds, ou plus généralement à estimer la probabilité d'existence d'une connexion entre deux entités du graphe. Cette tâche est particulièrement importante dans les graphes dynamiques, où l'objectif est d'anticiper l'évolution des interactions au cours du temps. C'est cette dernière famille de tâches qui nous intéresse particulièrement, puisque notre objectif est d'estimer, pour chaque valeur

de paramètre, la probabilité qu'elle apparaisse et qu'elle soit donc considérée comme légitime.

L'étendue des problèmes pouvant être modélisés sous forme de graphes, ainsi que la diversité des tâches applicables, expliquent la croissance rapide de la littérature sur ce sujet.

3.4.2 Etat de l'art des GNN

Nous analyserons désormais l'état de l'art concernant les GNN. Notre étude s'attachera à l'évolution historique de ces approches, en commençant par les GNN statiques. Nous examinerons ensuite les graphes dynamiques et les différentes architectures de GNN adaptées à la manipulation de ces types de graphes. Enfin, nous analyserons les approches non-supervisées, indépendamment de la dimension temporelle, afin d'évaluer la capacité des GNN à traiter des données non étiquetées.

3.4.2.1 GNN traditionnels

Le domaine des GNN est relativement récent et s'est fortement développé au cours de la dernière décennie. Le concept de réseaux de neurones sur graphes a été formellement introduit pour la première fois en 2008 par Scarselli et al. [?], qui ont proposé un framework pour l'apprentissage sur des données structurées en graphe, basé sur des mécanismes itératifs de propagation et d'agrégation d'états. Des travaux ultérieurs ont considérablement amélioré la praticabilité et la scalabilité de cette formulation initiale. En particulier, Kipf et Welling [?] ont introduit les Graph Convolutional Networks (GCN), qui simplifient le processus de propagation des messages en linéarisant l'agrégation de voisinage.

Dans leur formulation moderne, les GNN peuvent être vus comme réalisant des convolutions basées sur le voisinage, où l'information provenant des nœuds adjacents est agrégée afin d'enrichir la représentation de chaque nœud avec un contexte local. La plupart des architectures contemporaines de GNN suivent le paradigme de *message passing* proposé par Gilmer et al. [?], composé de trois phases principales : la construction des messages à partir des embeddings des nœuds, l'agrégation des messages provenant des voisins, et la propagation (ou mise à jour) de l'information agrégée pour produire de nouvelles représentations des nœuds.

Sur la base du cadre MPNN standard, plusieurs extensions ont été proposées. Par exemple, les Graph Attention Networks (GAT) [?] introduisent des mécanismes d'attention qui attribuent des poids adaptatifs aux nœuds voisins lors de l'agrégation. GraphSAGE [?] permet un apprentissage inductif sur des nœuds jusqu'alors non vus en échantillonnant un voisinage de taille fixe et en appliquant des fonctions d'agrégation apprenables telles que la moyenne, le pooling ou des agrégateurs basés sur LSTM. D'autres variantes notables incluent les Graph Isomorphism Networks (GIN) [?] qui utilisent une agrégation par somme injective et des MLP pour mieux distinguer la structure globale du graphe.

Cependant, la plupart de ces architectures de GNN sont principalement conçues

pour des graphes statiques. Or, nos données évoluent dans le temps et le graphe associé est donc dynamique. Nous nous intéressons donc aux modèles de graphes dynamiques, une classe encore plus récente de GNN. Parmi ces modèles, on distingue deux grandes catégories selon la granularité temporelle.

3.4.2.2 GNN Dynamiques

La première famille de graphes dynamiques, les réseaux de graphes à temps discret (DTGN), constitue l'approche la plus directe pour intégrer l'information temporelle. Dans ce cadre, l'évolution du graphe est modélisée comme une suite de snapshots de l'état du graph observés à intervalles réguliers. La dimension temporelle est ensuite prise en compte via des RNN appliqués en à posteriori en complément du GNN. C'est notamment le cas de T-GCN [?] et EvolveGCN [?]. Cette formulation permet d'adapter relativement facilement des architectures de GNN classiques à un contexte temporel. DySAT [?] va un cran plus loin et propose un mécanisme d'attention temporelle où chaque nœud est associé à ses représentations historiques afin de capturer l'évolution de son contexte au cours du temps. Bien que ces approches soient relativement simples à mettre en œuvre, elles présentent plusieurs limites. En particulier, le fait de reposer sur une représentation par snapshots implique de conserver l'état complet du graphe à plusieurs instants, ce qui engendre un coût mémoire et calculatoire élevé. De plus, ces méthodes peinent à capturer des dépendances temporelles longues.

La seconde catégorie correspond aux réseaux de graphes en temps continu, qui opèrent à une granularité temporelle plus fine que les approches discrètes. Plutôt que de reposer sur des instantanés fixes, ces modèles traitent des séquences d'événements correspondant à l'ajout, la suppression ou la modification de nœuds et d'arêtes dans le graphe. Cette modélisation événementiel permet d'intégrer directement les dépendances temporelles dans le modèle, plutôt que de les traiter de manière externe.

Parmi les modèles de graphes dynamiques en temps continu, on distingue généralement deux sous-familles : les modèles d'encodage temporel *Temporal Embedding* et ceux de de voisinage temporel *Temporal Neighborhood*. Les modèles *Temporal Embedding* ne stockent pas explicitement d'état associé aux nœuds et s'appuient principalement sur des architectures récurrentes ou attentionnelles pour capturer l'évolution temporelle à partir de la séquence des événements observés. Parmi eux, TGAT [?] adapte le mécanisme de GAT [?] au contexte temporel en intégrant un encodage du temps directement dans le mécanisme d'attention.

La plupart des GNN dynamiques reposent sur des stratégies d'échantillonnage multi-saut afin de récupérer un voisinage temporel étendu. Ce processus est explicite dans la majorité des approches, mais peut également être implicite dans les modèles utilisant un RNN. CAWs [?], au lieu de faire des parcours multi-saut classiques, propose une alternative fondée sur des parcours causaux qui exploite l'hypothèse de fermeture triadique, selon laquelle deux nœuds partageant un voisin commun ont davantage de chances d'être connectés. Cette hypothèse est cependant

moins adaptée à notre cas d'étude basé sur un graphe biparti.

Plus récemment, DyGFormer [?] simplifie cette approche en se limitant à un voisinage de distance 1, tout en ajoutant des informations de cooccurrence. Celles-ci sont calculées à partir de la fréquence d'apparition des nœuds dans les événements du voisinage temporel, puis injectées dans un Transformer. Les auteurs proposent également une implémentation unifiée facilitant la comparaison des modèles de graphes dynamiques. Comme DyGFormer, GraphMixer [?] se limite également à un voisinage de distance 1. Les auteurs critiquent l'utilisation systématique de RNN et de mécanismes d'attention dans les graphes dynamiques, en expliquant que ces architectures ne sont nécessaires que pour agréger efficacement des dépendances multi-saut. En se limitant à un voisinage local, GraphMixer remplace ces mécanismes par des architectures de type MLP-Mixer, beaucoup plus simples et rapides à inférer.

À l'inverse des modèles *Temporal Embedding*, les modèles *Temporal Neighborhood* maintiennent un état latent associé à chaque nœud déjà observé, mis à jour au fil des événements. Cette approche est plus coûteuse en mémoire et en calcul, mais elle permet de conserver un historique condensé des interactions passées et de fournir davantage de contexte aux RNN lors du traitement de nouvelles interactions. Du côté des graphes dynamiques continus avec mémoire, on retrouve notamment DyRep [?], qui distingue les événements de communication des événements d'association entre nœuds. Cette approche est particulièrement adaptée aux multigraphes dynamiques, mais elle est moins pertinente dans notre cas où il n'existe qu'une seule arête possible par paire de nœuds. JODIE [?] est quant à lui conçu pour la prédiction de liens sur graphes bipartis. Le modèle maintient pour chaque utilisateur et chaque item un embedding dynamique mis à jour par des RNN mutuellement récursifs, en complément d'une représentation statique. Enfin, TGN [?] propose une formalisation générale des graphes dynamiques continus avec mémoire. Le modèle introduit une architecture modulaire composée d'un mécanisme de mémoire, d'une fonction de mise à jour, d'un module de message passing et d'un encodeur temporel. De nombreux modèles existants peuvent ainsi être interprétés comme des cas particuliers de ce formalisme.

	Modèle	Attention	RNN	Mémoire	Parcours
Temporal Embedding	TGAT [?]	Oui	Non	Non	multi-saut
	CAWs [?]	Oui	Oui	Non	causal
	DyGFormer [?]	Oui	Non	Non	1-hop
	GraphMixer [?]	Non	Non	Non	1-hop
Temporal Neighborhood	DyRep [?]	Non	Oui	Oui	multi-saut
	JODIE [?]	Non	Oui	Oui	multi-saut
	TGN [?]	Oui	Oui	Oui	multi-saut

Table 3.2: Comparaison des principaux modèles de graphes dynamiques en temps continu

Une vue d'ensemble de l'apprentissage sur graphes dynamiques est proposée

par Feng et al. [?], qui présentent une synthèse mettant en évidence les différences conceptuelles entre les modèles dynamiques à temps discret et à temps continu, ainsi qu'un benchmark étendu des principales méthodes de ces deux catégories. Le tableau ?? fournit une comparaison synthétique de ces modèles selon leurs mécanismes d'attention, leur utilisation de RNN et de mémoire, ainsi que leur stratégie d'échantillonnage de voisinage.

3.4.2.3 GNN non supervisé

L'ensemble des modèles précédemment énoncés sont conçus pour pouvoir être utilisés sur des tâches de prédiction de liens, qui relèvent de l'apprentissage auto-supervisé. Ce cadre est généralement considéré comme une sous-catégorie de l'apprentissage non supervisé, dans lequel des pseudo-labels sont construits automatiquement à partir des données brutes. Dans le cas des graphes, cela consiste à opposer des arêtes positives, correspondant aux interactions réellement observées dans le graphe, à des arêtes négatives, générées artificiellement à partir de paires de nœuds n'ayant jamais été connectées. Ces arêtes sont encodées puis passées dans une couche fully connected qui prédit la probabilité d'existence de l'arête. La prédiction obtenue est ensuite comparée à la vérité afin de calculer la fonction de perte. Cette méthode fonctionne mais la génération d'arcs négatifs pertinents constitue un problème difficile. Il est en effet, à la fois possible de produire des arcs négatifs qui pourraient en réalité être légitimes, ou au contraire de ne pas couvrir correctement la diversité des comportements anormaux lorsque ceux-ci sont échantillonnés de manière aléatoire. Ce problème est d'autant plus marqué dans des contextes tels que le nôtre, où les sous-graphes sont fortement structurés et contraints.

L'autre approche non-supervisée couramment utilisée pour la détection d'anomalies consiste à formuler le problème comme une tâche de reconstruction des données. Pour ce faire, on utilise un autoencodeur qui prend en entrée un graphe et génère une représentation latente compressée à partir duquel il va essayer de recréer l'entrée d'origine. Après avoir été entraîné sur des données ne contenant pas d'anomalies, l'autoencodeur est capable de reconstruire correctement les points considérés comme normaux, mais devrait normalement échouer davantage sur des données atypiques qu'il n'a jamais observées auparavant. L'erreur de reconstruction peut alors être utilisée comme score d'anomalie. Plus cette erreur est élevée, plus le point est susceptible d'être anormal. C'est notamment le principe utilisé par DONE [?], qui repose sur deux autoencodeurs parallèles. Le premier encode la structure des liens du graphe, tandis que l'autre encode les attributs des nœuds. Les auteurs proposent également AdONE [?], une extension basée sur l'apprentissage adversarial permettant de construire des embeddings plus robustes aux valeurs aberrantes. GraphMAE[?] utilise également une architecture de type autoencodeur. Leur contribution principale consiste à appliquer un double masquage. Un premier masque est appliqué sur les attributs des nœuds avant l'encodage, puis un second est appliqué dans l'espace latent après l'encodage. Dans leur cas, l'encodeur est

un GNN, tandis que le décodeur est un MLP. Cette stratégie de masquage vise à renforcer la robustesse des représentations apprises en forçant le modèle à inférer des informations manquantes à différents niveaux de représentation. STGMAE[?] est presque identique mais enrichit l'entrée du GNN par une couche d'encodage spatio-temporel. Toutefois, ces approches restent limitées au cadre des graphes statiques et ne prennent pas directement en compte l'évolution temporelle de la structure du graphe. DyGIS[?] adapte justement GraphMAE aux graphes discrets dynamiques mais, à notre connaissance, aucun travail de l'état de l'art ne propose encore d'étendre les méthodes de reconstruction au cas des graphes dynamiques en temps continu non supervisé.

Parmi les deux approches d'apprentissage non-supervisées présentées, la prédiction d'arcs via l'auto-supervision semble plus facilement équilibrable, étant donné la nature binaire de la tâche. Cependant, la création d'arcs négatifs qui est nécessaire à l'auto-supervision est sujette à l'introduction de biais. En revanche, l'approche de reconstruction est moins susceptible à ces biais mais est plus difficile à guider.

3.4.3 Formalisation adapté à l'utilisation de GNN dynamiques

Pour considérer la dimension temporelle et détecter les attaques d'inondation ou à plusieurs étapes, il faut intégrer l'information temporelle à notre détection et donc considérer les graphes dynamiques. Une solution serait d'utiliser les graphes dynamiques discrets et c'est d'ailleurs l'approche qu'emploie GSAD [?].

Nous nous intéressons donc davantage aux graphes dynamiques continus, tels qu'introduits dans le cadre de TGN [?]. Dans cette approche, plutôt que de représenter le graphe comme un objet statique décrivant un état à un instant donné, celui-ci est modélisé comme une séquence d'événements qui le construit progressivement.

$$\mathcal{G} = \{e_1, e_2, \dots, e_n\}$$

où e_i désigne l'événement apparaissant à la i -ème position dans la séquence.

Ces événements correspondent généralement à des ajouts, suppressions ou modifications de nœuds et d'arêtes. Toutefois, notre cadre est plus simple car notre graphe est purement additif. Chaque arête qui serait ajoutée dans un graphe traditionnel est représentée comme un événement défini par cette arête ainsi que par les deux nœuds qu'elle relie.

$$e_i = (u, v_i, x_i),$$

où $u \in \mathcal{V}_c$ désigne un nœud central, $v_i \in \mathcal{V}_f$ un nœud de paramètre contenant la valeur de ce dernier et $\mathbf{z}_i \in \mathbb{R}^d$ le nom du paramètre contenu dans l'arête.

Cette formulation nécessite d'adapter les algorithmes classiques utilisés dans les GNN, mais elle permet une représentation beaucoup plus fine, dans laquelle aucune information temporelle n'est perdue. La figure ?? illustre la transformation d'une

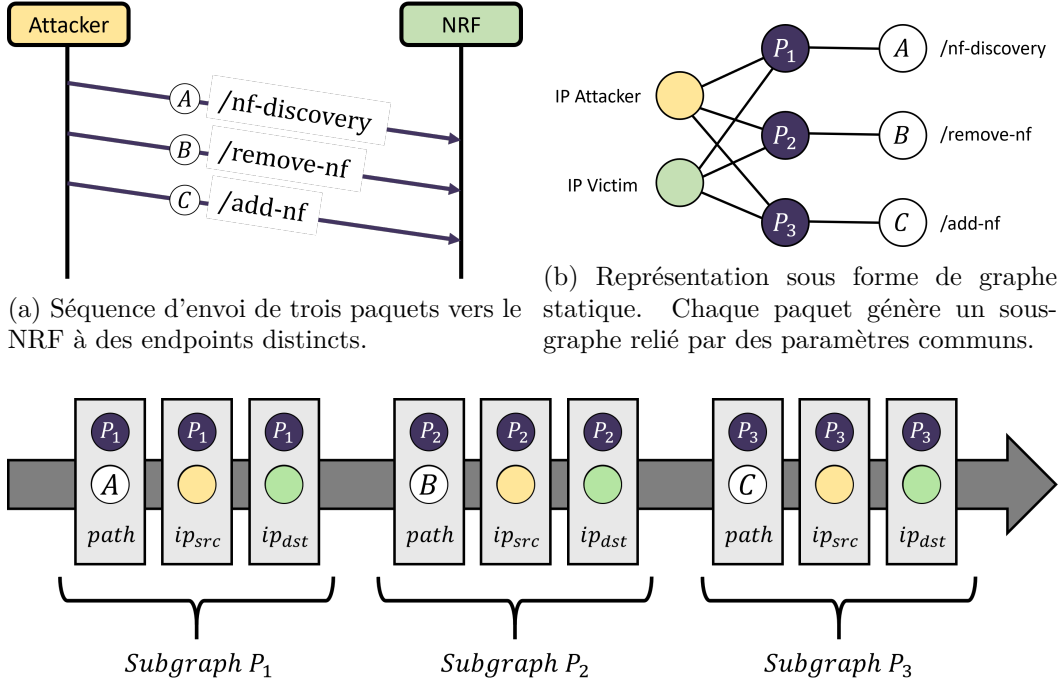


Figure 3.9: Différentes représentations d'une même séquence d'échanges entre un attaquant et le NRF : diagramme de séquence, graphe statique, puis graphe dynamique continu.

forme de graphe statique à celle d'un graphe dynamique continu. On observe que, pour les trois paquets, neuf arcs sont générés dans le graphe statique, lesquels se traduisent ensuite en autant d'événements dans le graphe continu.

3.4.4 Fonctionnement détaillé des GNN dynamique

Dans leurs travaux, TGN [?] proposent un cadre de formalisation des GNN dynamiques, permettant de représenter les travaux de l'état de l'art comme des spécifications de leur modèle. Nous nous appuyons sur ce cadre pour présenter et comprendre le fonctionnement d'un GNN dynamique. La figure ?? illustre les quatre modules principaux définis par TGN [?] : un module de mémoire, un module d'agrégation de messages, un module de mise à jour de la mémoire et un module d'inférence.

3.4.4.1 Création des messages

La première étape d'un GNN consiste à construire un message contenant l'ensemble des informations associées à un événement du graphe dynamique.

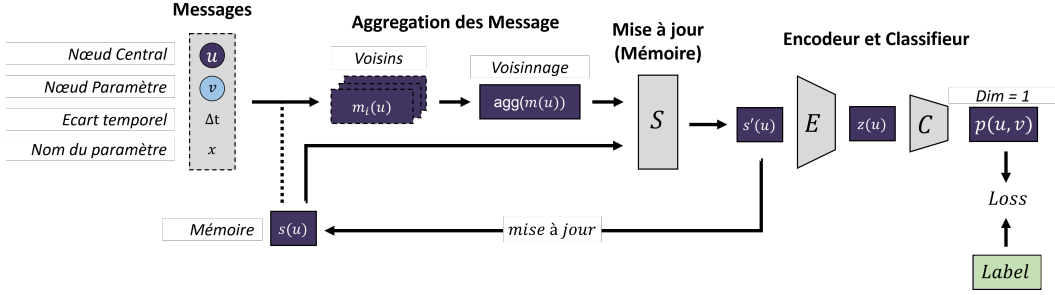


Figure 3.10: Vue d'ensemble de l'architecture générale d'un GNN Dynamique Continu, illustrant les quatre modules principaux : mémoire, agrégation de messages, mise à jour de la mémoire et inférence.

$$m_i = msg(s(u), s(v_i), x_i, \Delta t)$$

Comme expliqué dans la section 3.3.3, la valeur des noeuds paramètres ne nous intéressent pas. Les vecteurs associés aux noeuds centraux $s(u)$ et aux noeuds paramètres $s(v_i)$ sont donc initialisés comme des vecteurs nuls. Si ces noeuds ont déjà été impliqués dans des événements précédents, leurs représentations qui a déjà subi une convolution est alors réutilisées. Le vecteur associé à l'arête x_i est quant à lui déterminé à partir du nom du paramètre porté par l'événement. Dans notre cas, ce nom de paramètre correspond à un token textuel et il est donc nécessaire de l'encoder au préalable sous forme d'un vecteur numérique. La méthode utilisée pour cet encodage est présentée dans la section suivante. Il existe diverses méthodes pour définir l'intervalle temporel Δt mais la méthode la plus fréquemment employée consiste à calculer la différence temporelle entre l'échantillon et le dernier paquet encodé. Ce dernier est représenté sous forme de vecteur de dimensions identiques aux autres composantes du système. Un paramètre ajustable permet alors de moduler la granularité de cette mesure.

3.4.4.2 Agrégation des messages

L'étape d'agrégation consiste à considérer l'ensemble des voisins du nœud source en cours de traitement, c'est-à-dire tous les événements m_i précédant un temps t dans lesquels ce nœud apparaît. On obtient ainsi une collection de messages qu'on d'agrège ensuite afin de produire un vecteur unique représentant le voisinage de distance 1 du nœud considéré

$$\bar{m}_i(t) = \text{agg}(m_i(t_1), \dots, m_i(t))$$

Cette agrégation peut être réalisée de différentes manières. TGN [?] propose plusieurs stratégies, parmi lesquelles l'utilisation de RNN, d'un mécanisme d'attention, une moyenne des messages ou bien de simplement conserver l'événement le plus récent.

3.4.4.3 Mise à jour de la mémoire

Afin de prendre en compte la dimension temporelle, les modèles dynamiques de type *Temporal Neighborhood* disposent d'un module de mémoire, absent des modèles *Temporal Embedding*. Ce module combine le vecteur agrégé avec l'historique des représentations précédentes des éléments constitutifs du message. Cet ensemble est ensuite transmis à un RNN, qui joue le rôle de module de mémoire apprenable. Ce module produit un nouveau vecteur représentant à la fois le message et son historique.

$$s(u) = mem(x_i, s(u))$$

Cette nouvelle représentation est utilisée pour les modules futurs et remplace également l'ancienne valeur stockée en mémoire. TGN [?] propose notamment d'utiliser des architectures de type LSTM ou GRU pour implémenter ce module de mémoire.

3.4.4.4 Encodeur et Classifieur

L'encodeur est le module chargé de calculer la représentation des nœuds à partir de leur mémoire et des interactions temporelles. Plusieurs variantes peuvent être utilisées :

- **Identity (id)** : l'encodage correspond directement à la mémoire du nœud.
- **Time projection (time)** : la mémoire est modulée en fonction du temps écoulé depuis la dernière interaction, comme dans JODIE [?].
- **Temporal Graph Attention (attn)** : plusieurs couches de graph attention agrègent l'information provenant du voisinage temporel du nœud. À chaque couche, les voisins temporels sont récupérés puis combinés via un mécanisme de multi-head attention.
- **Temporal Graph Sum (sum)** : version plus simple et rapide où les informations des voisins temporels sont agrégées par somme puis projetées linéairement.

Pour les modèles *attn* et *sum*, un encodage temporel de type *Time2Vec* est utilisé afin de représenter les écarts temporels entre interactions. Ces méthodes permettent également de limiter le problème de *staleness* en mettant à jour les représentations grâce aux informations des voisins récents.

3.4.5 Adaptation à notre cas d'utilisation

Pour mieux répondre à notre problématique, nous proposons deux ajustements justifiés par notre cadre d'utilisation. Dans un premier temps, nous supprimons l'information temporelle explicite des messages. Dans un second temps, nous

adaptons le mécanisme d'agrégation standard en une version spécifique, conçue pour notre structure bipartite.

Le premier ajustement consiste à supprimer l'information temporelle explicite du modèle. Concrètement, cela se traduit par le retrait du terme Δt dans la formulation des messages. Dans notre contexte, l'information temporelle est déjà implicitement capturée par les séquences d'interactions lorsqu'elle est traitée par le RNN. La mention explicite du terme Δt est principalement pertinente pour identifier d'éventuelles pauses ou délais dans le trafic. Toutefois, cette information ne constitue pas un indicateur suffisamment représentatif d'une attaque et induit souvent, au contraire, des biais de surapprentissage. Pour cette raison, nous avons choisi de retirer cette composante temporelle de la formulation des messages.

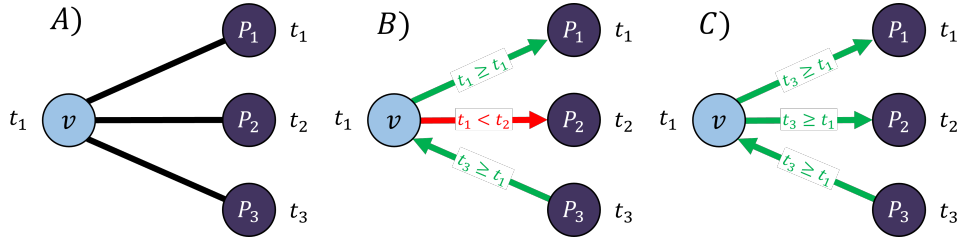


Figure 3.11: Représentation du problème lié au parcours multi-saut dans notre graphe biparti. La figure illustre (A) le graphe avec les arrivées temporelles des paquets, (B) la méthode classique de propagation multi-sauts contrainte temporellement, et (C) notre approche modifiée permettant de récupérer correctement les dépendances temporelles en propageant le temps du nœud initial.

Le deuxième ajustement concerne la construction des voisinages multi-sauts. Dans un GNN dynamique classique, un nœud ne peut agréger que des voisins apparus avant lui. Cependant, dans notre graphe bipartit, les nœuds paramètres sont réutilisés et conservent leur horodatage apparition. Cela crée un problème lors de l'exploration multi-sauts. Certains nœuds centraux, pourtant antérieurs au nœud cible ne sont pas récupérés, car ils sont postérieurs au nœud paramètre intermédiaire. Pour résoudre ce problème, nous propageons le temps d'arrivée du nœud cible lors de l'exploration du voisinage, plutôt que celui des nœuds intermédiaires. Cela permet de conserver un voisinage temporel cohérent malgré la réutilisation des nœuds paramètres. La problématique ainsi que solution que nous proposons sont représentés sur la figure ??.

Ces ajustements répondent directement aux spécificités de notre cas d'usage et ont donc été intégrés à notre approche. La suppression de l'information temporelle explicite permet de limiter les risques de surapprentissage liés aux irrégularités du trafic, tandis que la modification du mécanisme de propagation garantit une exploration multi-sauts cohérente dans le contexte particulier de notre graphe bipartit.

3.4.5.1 Problèmes ouverts

D'autres pistes d'ajustement ont également été envisagées au cours de ce travail, mais n'ont finalement pas été intégrées à l'approche proposée. Elles restent néanmoins intéressantes à mentionner, car leur considération est légitime et soulève plusieurs questions pertinentes quant à notre formalisation.

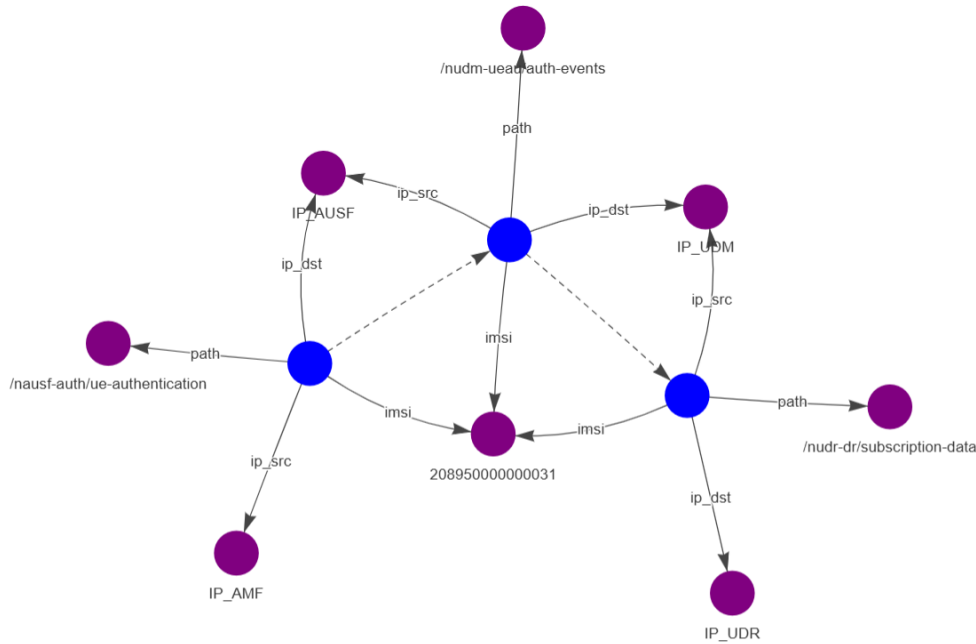


Figure 3.12: Représentation sous forme de graphe d'une procédure composée de trois paquets. Les arcs orientés en pointillés représentent un nouveau type de liaison entre nœuds central permettant de représenter l'ordre temporel des échanges.

Dans un réseau 5G, le déclenchement d'une procédure engendre la génération de séquences ordonnées de messages. Il peut être envisagé de représenter explicitement ces séquences ainsi que leur ordre au sein du graphe au moyen d'une formalisation dédiée. Une approche possible consiste à orienter le graphe et à introduire un type de liaison spécifique pour relier un nœud central à son successeur dans la séquence. La Figure ?? illustre l'ajout de dépendance par une flèche pointillée. Cette représentation présente l'avantage d'être aisément interprétable par un lecteur humain tout en rendant explicites les séquences et leur ordre d'exécution. Cependant, cette approche n'apporte pas d'information supplémentaire au modèle, dans la mesure où la structure séquentielle est déjà encodée via les graphes dynamiques continus. Néanmoins, elle conserve un intérêt pour l'analyse humaine et peut ainsi être ajoutée a posteriori de l'inférence afin d'améliorer la lisibilité des résultats.

EGAT[?] propose de donner plus d'importance à l'information portée par les arêtes en réalisant une agrégation sur ces dernières au lieu d'une agrégation sur les nœuds comme cela est fait classiquement. Pour ce faire, ils convertissent leur graphe

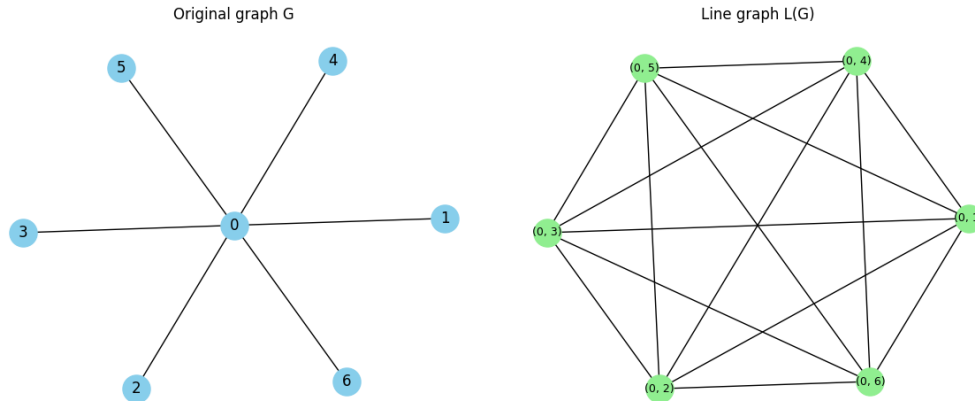


Figure 3.13: Illustration de la transformation d'un graphe classique (à gauche) en *line graph* (à droite). Dans cette représentation, les arêtes du graphe initial deviennent les nœuds du *line graph*, tandis que les relations entre arêtes sont représentées par de nouvelles connexions.

en *line graph* où les nœuds correspondent aux arêtes du graphe original, tandis que les arêtes traduisent les relations entre les noeuds. La Figure ?? illustre cette transformation. Néanmoins, dans notre cas d'étude, les graphes considérés sont fortement connectés et comportent un grand nombre d'arêtes. Or, la construction du *line graph* entraîne une augmentation significative de la complexité structurelle, puisque chaque arête du graphe initial devient un nœud supplémentaire. Cette transformation ne passe donc pas à l'échelle dans notre configuration et conduit rapidement à une explosion de la complexité.

Ces différentes approches montrent que plusieurs extensions de la formalisation initiale sont envisageables. Néanmoins, bien qu'elles présentent des avantages évidents, elles soulèvent des problèmes structurels et computationnels qui empêchent leur intégration à notre formalisation.

3.5 Encodage des paramètres de dimension non-finie via NLP

Comme mentionné dans la section relative à la construction des messages, il est nécessaire d'encoder les informations contenues dans les arcs. Les paquets réseaux 5G comportent un très grand nombre de paramètres, ce qui conduit à un espace d'entrée de très grande dimension. Dans le cadre d'une approche de détection de type *zero-day* basée sur l'apprentissage non supervisé, il est essentiel de pouvoir représenter l'ensemble des attributs potentiellement observables, dans la mesure où il est impossible d'anticiper a priori lesquels pourront être exploités par de futures attaques. Cette situation implique la manipulation d'un espace de représentation de très grande dimension, susceptible d'entraîner le phénomène de *malédiction de la dimension*, qui se caractérise par une dégradation de la généralisabilité lorsque la

dimension de l'entrée est trop grande. Par conséquent, il est nécessaire de réduire la dimension de cet espace d'entrée afin de le rendre exploitable par les modèles d'apprentissage GNN mentionnés précédemment.

3.5.1 Tour d'horizon des techniques de NLP

Pour répondre à ce problème, nous nous sommes intéressés à des approches issues du traitement automatique du langage naturel (NLP), traditionnellement utilisées pour des tâches telles que la traduction automatique, la classification de documents, la détection d'intention ou encore la reconnaissance d'entités nommées. Dans ces applications, les modèles sont entraînés à apprendre une représentation sémantique des données textuelles, sous la forme d'un espace latent structuré dans lequel une proximité géométrique reflète une proximité sémantique. Dans ce travail, nous cherchons à exploiter directement la structure de l'espace latent afin d'associer à n'importe quel paramètre une représentation vectorielle pertinente.

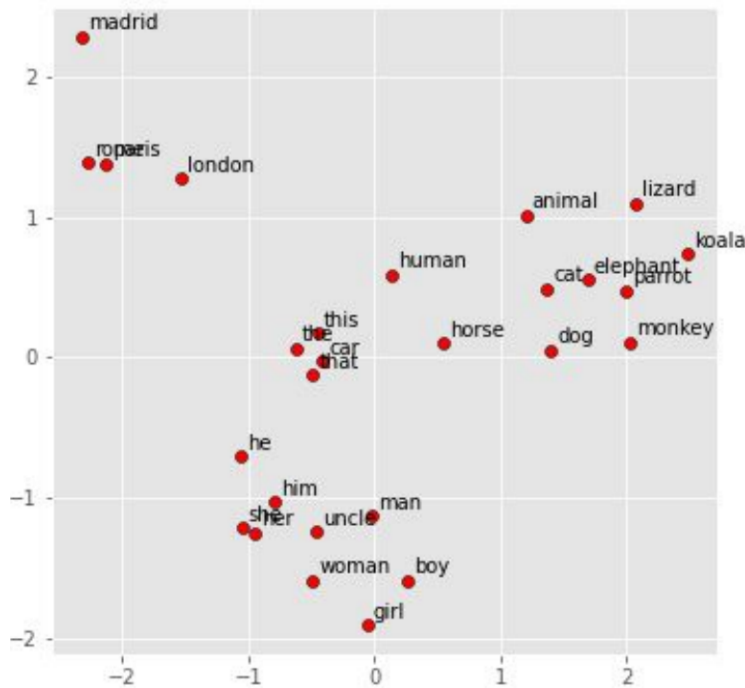


Figure 3.14: Projection en deux dimensions d'un espace latent représentant un vocabulaire anglais, obtenue par analyse en composantes principales (PCA).

Pour évaluer la qualité de cette structuration, deux approches complémentaires sont généralement utilisées et reposent sur une interprétation humaine. La première consiste à examiner, au cas par cas, les voisins les plus proches d'un mot donné dans l'espace d'embedding. La seconde consiste à projeter cet espace de haute dimension dans un espace à deux dimensions afin d'en permettre une visualisation globale. Les méthodes les plus couramment utilisées pour cela sont la PCA et le t-SNE, qui

représentent respectivement une projection linéaire conservant la structure globale, et une projection stochastique visant à préserver les similarités locales. La figure ?? illustre un exemple de projection PCA d'un vocabulaire générique encodé avec FastText[?] que nous allons présenter ensuite.

Notre objectif est d'encoder les mots de manière individuelle. L'approche la plus simple consiste à utiliser un encodage one-hot, c'est-à-dire associer chaque mot à une dimension d'un vecteur orthogonal. Le fait d'attribuer des représentations uniques à chaque mot laisse ensuite au GNN la responsabilité d'en apprendre la sémantique. Un inconvénient de cette méthode est qu'elle nécessite de partir d'embeddings sans sémantique. On pourrait alors envisager de faciliter l'apprentissage du GNN en fournissant une initialisation plus informative des paramètres à l'aide de méthodes issues du NLP. Un autre inconvénient majeur est que la construction de vecteurs orthogonaux impose une dimension de l'espace d'embedding égale à la taille du vocabulaire.

Afin de pallier ce problème, des techniques comme le hashing trick projettent les termes dans un espace de dimension fixe à l'aide d'une fonction de hachage. Cette approche permet de contrôler la taille des représentations indépendamment du vocabulaire, mais introduit des collisions. Ainsi, plusieurs termes distincts peuvent obtenir des encodages similaires, ce qui peut induire des proximités artificielles dans l'espace vectoriel. Le hashing trick et le one-hot encoding ont l'avantage d'être simples et rapides à grande échelle, mais ne capturent pas la dimension sémantique des mots qu'ils encodent.

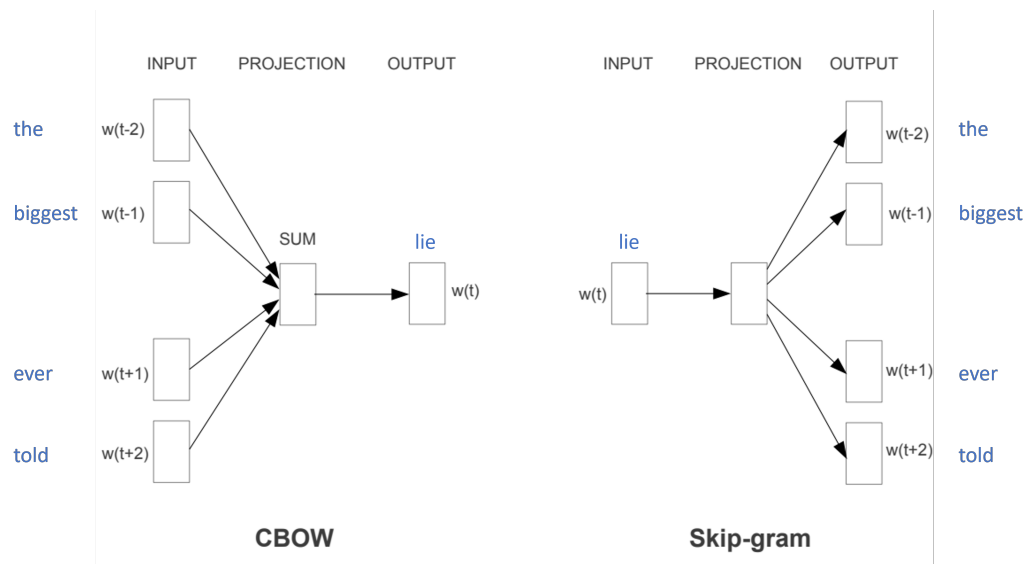


Figure 3.15: Illustration de l'architecture de Word2Vec à travers les modèles Continuous Bag of Words (CBOW) et Skip-Gram, appliqués au mot cible *lie* dans la phrase *The biggest lie ever told*.

Les approches de deep learning permettent justement d'apprendre la sémantique des mots à partir de leur contexte d'utilisation. L'idée principale est qu'un mot

peut être compris en observant les mots qui l'entourent dans une phrase ou un corpus. C'est ce que fait Word2Vec[?] qui propose deux modèles, un Continuous BoW (CBOW) qui vise à prédire un mot à partir de son contexte et un Skip-Gram qui, de façon symétrique, vise à prédire les mots du contexte en connaissant un mot en entrée.

Néanmoins, comme il apprend sur des fenêtres de contexte locales, Word2Vec ne dispose pas d'une vision globale du corpus. Cette limitation est adressée par GloVe (Global Vectors for Word Representation) [?], qui apprend à minimiser la distance entre le produit scalaire de deux vecteurs et le logarithme de leur fréquence de co-occurrence dans le corpus. Cependant, GloVe présente des limites dans le cas de langues à morphologie riche, ainsi que pour les mots rares.

FastText [?] est justement une extension de Word2Vec qui cherche à découper un mot en sous-unités morphologiques. Contrairement à ce Word2vec, FastText ne considère pas un mot comme une entité atomique, mais le représente comme un ensemble de n -grammes de caractères. Cette segmentation permet de partager des informations entre mots présentant des structures similaires. Dans notre cas, cela permet par exemple de décomposer *servingNfId* en plusieurs sous-unités telles que *serving*, *Nf* et *Id*. Chacun de ces segments peut alors porter une information sémantique propre et être réutilisée pour représenter d'autres paramètres partageant des sous-parties communes. Cependant, une limite importante de cette approche réside dans le caractère statique des embeddings. Chaque mot et n -gramme possède une représentation fixe, indépendante du contexte dans lequel il apparaît.

Or, un mot peut être polysémique et donc avoir une sémantique variable en fonction de son contexte d'apparition. Afin de prendre en compte cette ambiguïté sémantique, on a vu l'arrivée de travaux utilisant des architectures de type Transformer, qui permettent d'apprendre des représentations contextuelles dynamiques. Ces modèles, tels que GPT [?] et BERT [?], utilisent des mécanismes d'attention pour modéliser les dépendances à long terme dans les séquences de texte, ce qui améliore significativement la qualité des représentations apprises. Le premier est basé sur un décodeur qui va analyser le contexte historique d'un mot pour prédire sa suite tandis que le second va prendre le contexte bidirectionnel et sera plus adapté à des tâches de compréhension du langage. Ces modèles sont la plupart du temps pré-entraînés sur de très grands corpus de texte, mais peuvent être adaptés à des tâches spécifiques par le biais d'une phase de fine-tuning. Cependant, leur complexité et leurs besoins en ressources computationnelles sont considérablement plus élevés que les modèles précédents.

Parmi les approches proposées, GPT, BERT et leurs variantes respectives figurent parmi les modèles obtenant généralement les meilleures performances. Néanmoins, il convient de souligner que ces architectures ont été conçues pour des données textuelles naturelles et reposent sur des hypothèses implicites liées à la structure du langage humain. Dans le cadre de ce travail, le contexte considéré est sensiblement différent. Le vocabulaire manipulé est très spécifique, parfois constitué d'identifiants ou de chaînes non lexicalisées, et les paquets qui les contiennent diffèrent fortement de la structure d'une phrase naturelle. Dès lors, l'utilisation de

modèles aussi complexes peut s'avérer disproportionnée au regard de la nature des données. Il apparaît donc nécessaire d'évaluer dans quelle mesure ces architectures peuvent être adaptées à ce type de données, et d'identifier les modèles offrant le meilleur compromis entre expressivité, robustesse et coût computationnel dans un contexte aussi spécifique.

3.5.2 Mise en place de l'encodage sémantique

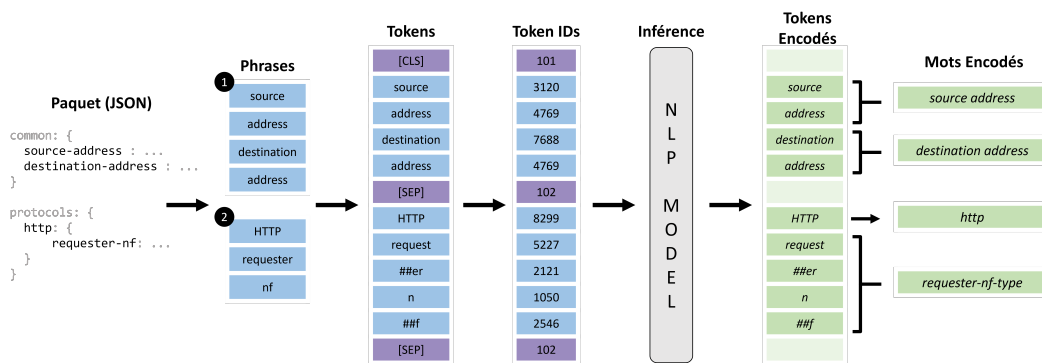


Figure 3.16: placeholder

La figure ?? illustre le processus d'application du NLP à notre problème afin d'obtenir des encodages associés aux mots. Dans un premier temps, nous exploitons le fichier JSON contenant les données extraites des paquets, que nous transformons en phrases. Pour cela, les chaînes de caractères sont nettoyées en supprimant les caractères spéciaux, tels que les tirets ou les points. Pour chaque couche du paquet, une phrase distincte est construite, et le nom du protocole est explicitement ajouté afin d'enrichir le contexte lexical. Le modèle considère ainsi l'ensemble du paquet comme une unité globale, tout en tirant parti de la séparation entre les phrases pour structurer l'information.

Ensuite, comme dans un pipeline NLP classique, une étape de tokenisation est appliquée. Celle-ci consiste à identifier des mots connus dans le vocabulaire et, lorsqu'un mot n'est pas reconnu, à le décomposer en sous-unités. Comme illustré sur la figure, le mot *requester* est par exemple segmenté en *request* et *##er*, ce dernier étant marqué comme suffixe par la présence des dièses. De même, le terme *nf*, inconnu du vocabulaire, est décomposé en *n* et *##f*.

Les tokens obtenus sont ensuite convertis en identifiants (token IDs), qui sont fournis au modèle. Celui-ci produit en sortie des vecteurs encodés, représentant chaque token. Comme un mot peut être composé de plusieurs tokens, il est nécessaire d'agréger ces représentations. La méthode la plus simple consiste à effectuer la moyenne des encodages des tokens associés à un même mot.

3.5.3 Avantages et limites

Le fait d'obtenir une proximité géométrique entre des éléments présentant une similarité sémantique constitue une propriété particulièrement intéressante dans notre contexte. En effet, de nombreux paramètres présentent une forte redondance lexicale ou fonctionnelle, comme *servingNfId* et *nf-id*. Ces paramètres peuvent ainsi être associés à des représentations proches dans l'espace latent, ce qui contribue à réduire la complexité du vocabulaire. De plus, l'utilisation de techniques de NLP pour enrichir la représentation sémantique des vecteurs d'arêtes peut fournir une information de départ supplémentaire au GNN, facilitant ainsi l'apprentissage de relations complexes.

Néanmoins, plusieurs limites apparaissent dans ce cadre. Pour réaliser l'embedding d'éléments textuels par NLP, il est possible soit d'utiliser directement des modèles pré-entraînés, soit de recourir à un fine-tuning afin de les adapter au vocabulaire spécifique du domaine. Dans notre cas, les paramètres à encoder s'apparentent davantage à des noms de variables qu'à des tokens issus du langage naturel. Par exemple, certains termes tels que *address* peuvent être correctement interprétés par les modèles, tandis que des identifiants techniques comme *avType* ou *kausf*, y compris leurs sous-unités, ne correspondent pas à des unités lexicales courantes.

Dans ce contexte, le recours à une phase de fine-tuning apparaît nécessaire. Celle-ci consiste à fournir au modèle des exemples de phrases afin d'apprendre la sémantique des tokens à partir de leur contexte d'occurrence. Toutefois, dans notre cas, les données que l'on extrait des paquets ne sont pas naturellement structurées sous forme de phrases. La solution que nous considérons est donc de représenter chaque paquet comme une phrase en concaténant ses différents paramètres. Cette approche demeure cependant imparfaite, car elle introduit implicitement une notion d'ordre et de position dans la séquence, laquelle n'a pas de signification intrinsèque dans notre problématique.

Que ce soit pour FastText, GPT ou BERT, l'ordre des mots dans une phrase est généralement centrale dans les mécanismes d'apprentissage. Il est donc difficile de prédire dans quelle mesure ces modèles parviendront à capturer efficacement la sémantique de notre vocabulaire dans un tel contexte. Par conséquent, nous évaluerons empiriquement, dans la section expérimentale présentée au chapitre suivant, les différents impacts des méthodes d'encodage afin d'identifier ceux qui permettent d'améliorer les performances de détection.

3.6 Conclusion

Nous avons vu dans ce chapitre qu'il est possible de réaliser de la détection d'anomalies en mode non supervisé, notamment à l'aide de GNNs reposant sur des tâches de reconstruction ou de prédiction de liens qui exploitent des arêtes positives et négatives. À première vue, cette approche non-supervisée peut sembler plus intéressante, car elle permet théoriquement de détecter des attaques

de type zero-day. Cependant, sa mise en œuvre est complexe pour plusieurs raisons. Tout d’abord, les approches de reconstruction ou de prédiction de lien auto-supervisé nécessitent un équilibre délicat entre les différents types d’arêtes, notamment les arêtes négatives, ce qui rend l’entraînement difficile à stabiliser. Par ailleurs, une approche non-supervisée implique souvent de considérer l’ensemble des caractéristiques d’un paquet, y compris celles qui ne sont pas pertinentes pour les attaques connues, ce qui introduit un bruit important dans les données. Ce bruit peut être partiellement réduit à l’aide de techniques issues du NLP, mais ces dernières introduisent elles-mêmes une incertitude supplémentaire, leur évaluation restant difficile à isoler du reste du modèle.

En comparaison, l’apprentissage supervisé présente certains avantages. Bien qu’il soit limité aux attaques connues, il est généralement plus simple à entraîner et à stabiliser. De plus, lorsque l’ensemble des attaques à détecter est défini à l’avance, il est possible de sélectionner un sous-ensemble de paramètres pertinents, réduisant ainsi la dimension du problème. En pratique, cette réduction de complexité contribue souvent à de meilleures performances. Dans notre cas, l’objectif ne se limite pas à la détection et inclut également l’interprétabilité. Comme l’approche supervisée reste explicable à travers notre formalisme, elle constitue également une piste tout à fait envisageable. Il existe ainsi plusieurs méthodes, chacune présentant ses avantages, nous proposons dans le chapitre suivant une analyse empirique afin d’évaluer les résultats de ces différents modèles.